

REDROB AI HACKATHON

Intelligent Candidate Discovery & Ranking Challenge

Complete Implementation Blueprint

Full Logic, Architecture & Scoring Design

(No UI or Boilerplate Code — Pure Implementation Logic)

0. Quick Summary — What This Document Covers

This document is your end-to-end implementation blueprint. It tells you WHAT logic to build, WHY each piece exists, and HOW each component connects. It does not include UI, boilerplate, or generic coding patterns — only pure decisions and logic you need to implement.

Document Section	What You Get
1 — Problem Deconstruction	What the judges actually measure, hidden traps, scoring formula
2 — Dataset Deep Dive	All 7 data files explained, schema fields, signal meanings
3 — Architecture Overview	Module map, data flow, file structure
4 — JD Parser Logic	How to extract requirements from job description
5 — Data Loader & Preprocessor	Streaming 100K candidates, field normalisation
6 — Honeypot Detector	Exact rules to identify ~80 impossible profiles
7 — Feature Engineering	Every feature you must compute with exact formulas
8 — Skill Scorer	Core skills list, proficiency weights, endorsement trust, assessment integration
9 — Career Analyzer	Product vs consulting, career trajectory, production signals
10 — Behavioral Signal Scorer	All 23 signals, weights, multiplier formula
11 — Final Scoring Engine	Composite formula, weights, how to combine all features
12 — Reasoning Generator	Template logic for 1-2 sentence per-candidate reasoning
13 — Submission Requirements	Exact CSV format, validation rules, what to submit
14 — Pre-computation Strategy	What to compute offline vs online, time budget allocation

1. Problem Deconstruction — What You Are Really Building

1.1 The Core Task

You have 100,000 candidate profiles. You have one job description for a Senior AI Engineer role at Redrob AI. You must output the top 100 best-fit candidates, ranked from best (#1) to 100th best, each with a score (0 to 1, non-increasing) and a 1-2 sentence human-readable reasoning.

KEY INSIGHT: This is NOT a keyword search problem. The judges have explicitly stated that finding candidates whose skills section contains the most AI keywords is the **WRONG** approach. You need to reason about the gap between what the JD says and what it means.

1.2 How Your Submission Is Scored

The final composite score formula (memorize this — it drives all your design decisions):

$$\text{Final Score} = 0.50 \times \text{NDCG@10} + 0.30 \times \text{NDCG@50} + 0.15 \times \text{MAP} + 0.05 \times \text{P@10}$$

Metric	Weight	What It Measures	Design Implication
NDCG@10	50%	Quality of your TOP 10 candidates	TOP 10 must be near-perfect fits — extremely high precision needed
NDCG@50	30%	Quality of your TOP 50 candidates	Top 50 must be consistently good — no bad picks in first half
MAP	15%	Average precision across all relevant candidates	You should not miss obvious good fits anywhere in your 100
P@10	5%	Precision in top 10	Same region as NDCG@10 — reinforces need for perfect top 10

DESIGN PRIORITY: Because NDCG@10 is 50% of your score, your TOP 10 picks matter far more than picks at rank 51-100. Build your logic to be extremely conservative and precise at the top, then looser as rank increases.

1.3 The Three Hidden Traps in the Dataset

The hackathon has explicitly built traps. Failing to handle them will destroy your score.

Trap Name	What It Is	How It Hurts You	How to Detect/Handle
Keyword Stuffers	Candidates with perfect AI skills list (Python, LLMs, embeddings, vectors) but whose actual job title is 'Marketing Manager' or 'Accountant'	Naive keyword matchers will rank these #1-5 and get terrible NDCG@10	Cross-check: skills must appear in career_history descriptions. Title must be ML/AI-adjacent.

Trap Name	What It Is	How It Hurts You	How to Detect/Handle
Honeypot Candidates (~80)	Profiles with impossible data: 8 years experience at a 3-year-old company, expert skill with 0 months usage, education dates impossible	If >10% of your top 100 are honeypots: AUTOMATIC DISQUALIFICATION	Run a validation pass: check timeline consistency, skill proficiency vs duration, company age vs experience
Inactive Perfect-Paper Candidates	Candidates with exactly the right skills and experience but who have last_active_date = 180+ days ago, recruiter_response_rate = 0.02, open_to_work_flag = false	You rank them #3, recruiter tries to contact them, they never respond — bad signal	Apply behavioral signal multiplier that heavily penalizes inactive candidates

1.4 The Plain-Language Fits (Positive Trap)

The JD says: 'A Tier 5 candidate may not use the words RAG or Pinecone in their profile, but if their career history shows they built a recommendation system at a product company, they are a fit.'

POSITIVE TRAP: Good candidates who describe their work in plain language (built a search system, designed retrieval pipeline, worked on recommendation engine) are real fits. Pure keyword matching misses them. You **MUST** use semantic/text matching on career_history descriptions.

1.5 The Explicit Disqualifiers from the JD

These are hard filters — any candidate matching these should receive a penalty large enough to push them out of top 100:

Disqualifier	How to Detect	Penalty Logic
Entire career at consulting firms ONLY (TCS, Infosys, Wipro, Accenture, Cognizant, Capgemini)	Check ALL entries in career_history.company against a consulting firm list	If ALL roles are at consulting firms: apply -0.4 penalty to final score. If some consulting + some product company: no penalty
Pure research roles with no production deployment	Check career_history titles for 'Research Scientist', 'Researcher' with no PM/Engineer indicators; check descriptions for deployment signals	If ALL roles are pure research (no production keywords): apply -0.3 penalty
AI/ML experience is ONLY recent LangChain/API	Check skill duration_months; check career descriptions for	If no pre-LLM AI experience: downgrade skills score by 50%

Disqualifier	How to Detect	Penalty Logic
wrappers (<12 months)	depth vs API calls	
Current 'tech lead' / 'architect' role with no coding for 18+ months	Title contains 'Architect', 'Tech Lead', 'VP of Engineering' with career descriptions showing no hands-on coding	Apply -0.2 penalty to career score
Computer Vision / Speech / Robotics only, no NLP/IR	Dominant skills are CV/Speech/Robotics; no NLP, search, retrieval, recommendation skills	Apply -0.25 to skills score if CV/Speech/Robotics dominant with no NLP/IR

2. Dataset Deep Dive — Every File Explained

2.1 File Inventory

File	Size	Purpose	When You Use It
candidates.jsonl	~465 MB	FULL 100K candidate pool — your main input	At runtime (streaming) and for offline pre-computation
sample_candidates.json	~300 KB	50 sample candidates in JSON array format	For testing and debugging your logic
candidate_schema.json	9 KB	JSON Schema defining all fields and types	Reference when building your data models
job_description.docx	40 KB	The actual job description to match against	Parse once at startup to extract requirements
redrob_signals_doc.docx	37 KB	Explanation of all 23 behavioral signals	Reference for signal scoring logic
submission_spec.docx	42 KB	Submission rules and evaluation details	Read carefully — all rules are there
sample_submission.csv	9 KB	Format reference for output CSV	Template for your output
validate_submission.py	5 KB	Validation script for your output	Run before every submission
submission_metadata_template.yaml	5 KB	Template for submission metadata	Fill before final submission

2.2 Candidate Data Structure — Full Field Reference

Each candidate in candidates.jsonl has this structure (6 top-level sections):

Section A: profile (basic info)

Field	Type	What to Use It For
candidate_id	string (CAND_XXXXXXX)	Primary key — required in output
headline	string	Quick classification signal — title-like, use for rapid filtering
summary	string (multi-sentence)	Rich text for semantic matching — embed this for similarity
location	string	City/region — for location scoring
country	string	Country — India preferred for this role
years_of_experience	number 0-50	Experience score input — target range 5-9 years
current_title	string	Job title — critical for filtering keyword stuffers

Field	Type	What to Use It For
current_company	string	For consulting firm detection
current_company_size	enum	Larger companies often = more production experience
current_industry	string	Industry relevance signal

Section B: career_history (array, up to 10 entries)

Field	Type	What to Use It For
company	string	Consulting firm detection, product company detection
title	string	Role seniority and function classification
start_date / end_date	date strings	Timeline validation (honeypot detection), experience calculation
duration_months	integer	Cross-validate with dates — mismatches = honeypot signal
is_current	boolean	Current role identification
industry	string	Tech/product vs services industry signal
company_size	enum	Scale of experience — larger = more production exposure
description	string	MOST IMPORTANT TEXT FIELD — embed for semantic matching of production AI/ML experience

Section C: education (array, up to 5 entries)

Field	Type	What to Use It For
institution	string	Less important for this role — JD is experience-focused
degree	string	CS/Engineering preferred but not required
field_of_study	string	Computer Science, AI/ML, Statistics relevant
start_year / end_year	integers	Timeline validation — honeypot detection
tier	enum (tier_1 to tier_4, unknown)	Mild positive signal — tier_1 from IITs/NITs/BITS; not a hard filter

Section D: skills (array)

Field	Type	What to Use It For
name	string	Match against JD required/preferred skills list
proficiency	enum: beginner/intermediate/advanced/expert	Quality signal — but TRUST IT ONLY if validated by duration + assessment + endorsements
endorsements	integer	Social proof — higher = more trustworthy claim

Field	Type	What to Use It For
duration_months	integer	Time with skill — critical for honeypot detection and trust scoring

CRITICAL SKILL TRUST FORMULA: $\text{Trust}(\text{skill}) = \min(1.0, (\text{duration_months}/24) * \text{proficiency_weight} * (1 + \text{endorsements}/50) * \text{assessment_bonus})$. A skill claiming 'expert' proficiency with 0 months and 0 endorsements should have near-zero trust.

Section E: certifications & languages

Certifications: mild positive signal if they have cloud/ML platform certs (AWS ML, GCP ML, Azure AI).
Languages: English is assumed required; not a differentiating factor for this role.

Section F: redrob_signals (23 behavioral signals)

Full signal reference (all 23 signals, grouped by importance for this specific JD):

Signal	Range	Importance	How to Use
open_to_work_flag	bool	HIGH	If false: multiply final score by 0.5. They may not be available.
last_active_date	date	HIGH	Days since last login. >180 days: multiply by 0.4. 30-180 days: multiply by 0.7. <30 days: no penalty.
recruiter_response_rate	0.0-1.0	HIGH	Direct availability signal. <0.1: multiply by 0.5. 0.1-0.4: no change. >0.4: small bonus (x1.1)
notice_period_days	0-180	MEDIUM-HIGH	JD wants sub-30 days. 0-30: bonus (x1.1). 31-60: no change. 61-90: mild penalty (x0.9). 91+: penalty (x0.8)
skill_assessment_scores	dict: skill→0-100	MEDIUM-HIGH	Platform-verified scores. If candidate claims advanced skill but scores <40: reduce skill trust. If >70: boost trust
avg_response_time_hours	number	MEDIUM	Lower is better. >72 hours: mild penalty. <12 hours: small bonus
interview_completion_rate	0.0-1.0	MEDIUM	<0.4: penalty (x0.85). >0.8: bonus (x1.05)
offer_acceptance_rate	0.0-1.0 or -1	-1 means no history	>0.5: positive signal. <0.2: mild concern. -1:

Signal	Range	Importance	How to Use
			neutral (no history)
github_activity_score	-1 to 100	MEDIUM	-1: no GitHub (neutral). >70: bonus for this AI/ML role (x1.1). <20: mild concern
profile_completeness_score	0-100	LOW-MEDIUM	<50: mild penalty. >80: no special action
saved_by_recruiters_30d	integer	LOW	>5 saves: small positive signal. Used as soft confirmation only.
willing_to_relocate	bool	MEDIUM	JD is Pune/Noida. If not in these cities and willing_to_relocate=false: apply location penalty.
preferred_work_mode	onsite/hybrid/remote/flexible	LOW	JD is hybrid. flexible/hybrid: no penalty. remote only: very mild penalty.
verified_email + verified_phone	bool	LOW	Both true: slight trust bonus. Used as final tiebreaker only.
linkedin_connected	bool	LOW	Small positive signal. Not a differentiator.
expected_salary_range_inr_lpa	min/max numbers	LOW	Very high expectation (>50 LPA for a startup Series A) could signal poor culture fit. Note it in reasoning.
profile_views_received_30d	integer	LOW	Passive signal — shows market demand for this person. Mild positive if >20.
applications_submitted_30d	integer	LOW	Shows active job search. >5 may indicate instability or desperation — neutral signal.
search_appearance_30d	integer	LOW	Recruiters finding them often = their profile is strong. Mild positive.
connection_count	integer	LOW	Social proof but not highly relevant. Use as very weak tiebreaker.
endorsements_received	integer	LOW	Total endorsements. Part of skill trust calculation only.
signup_date	date	INFO ONLY	Not a scoring factor. Use for context.

Signal	Range	Importance	How to Use
applications_submitted_30d	integer	LOW	Active job seeker signal. Mild positive.

3. Architecture Overview — Module Map

3.1 System Flow Diagram

Your system has two phases: OFFLINE (pre-computation, run once before submission) and ONLINE (actual ranking, must complete in 5 minutes on CPU).

Phase	Step	Module	Output
OFFLINE	1	jd_parser.py	requirements.json (parsed JD requirements, skill lists, weights)
OFFLINE	2	precompute_embeddings.py	candidate_texts.pkl (embedded summaries + career descriptions for all 100K)
OFFLINE	3	precompute_features.py (optional)	features_cache.pkl (all computed features for 100K candidates)
ONLINE	4	data_loader.py	Stream candidates from candidates.jsonl, yields parsed candidate dicts
ONLINE	5	honeypot_detector.py	honeypot_flags.json (flag True/False per candidate)
ONLINE	6	feature_extractor.py	Per-candidate feature vector (dict of all scored dimensions)
ONLINE	7	scoring_engine.py	Per-candidate composite score (float 0-1)
ONLINE	8	ranker.py	Sorted top-100 list with scores
ONLINE	9	reasoning_generator.py	1-2 sentence reasoning per top-100 candidate
ONLINE	10	output_writer.py	submission.csv (final output, validate with validate_submission.py)

3.2 File Structure

Recommended project structure:

rank.py	← Main entrypoint (the command judges will run)
config.py	← All weights, thresholds, skill lists – single source of truth
jd_parser.py	← Parse job_description.docx into requirements dict
data_loader.py	← Stream candidates.jsonl efficiently
honeypot_detector.py	← Validate timeline/proficiency consistency
feature_extractor.py	← Compute all feature vectors per candidate
skill_scorer.py	← Skill matching and trust calculation
career_analyzer.py	← Career history analysis and quality scoring
behavioral_scorer.py	← Redrob signals processing
scoring_engine.py	← Final composite score from all features
reasoning_generator.py	← Generate per-candidate reasoning text

<code>output_writer.py</code>	← Write valid submission CSV
<code>precompute_embeddings.py</code>	← Offline: embed all candidate texts
<code>precompute_features.py</code>	← Offline (optional): cache all features
<code>requirements.txt</code>	← Python dependencies
<code>submission_metadata.yaml</code>	← Filled submission template
<code>README.md</code>	← Setup and reproduce instructions

4. JD Parser Logic — Extracting Requirements

4.1 What to Extract from the Job Description

The JD parser runs once and produces a structured requirements object that all other modules consume. You need to extract:

4.1.1 Hard Required Skills (must have)

Skill Group	Primary Keywords	Semantic Equivalents (accept these too)
Embeddings / Vector Search	sentence-transformers, OpenAI embeddings, BGE, E5, embedding drift, index refresh	vector representations, dense retrieval, semantic search, bi-encoders, cross-encoders
Vector Databases	Pinecone, Weaviate, Qdrant, Milvus, FAISS, Elasticsearch, OpenSearch	vector store, approximate nearest neighbor, ANN index, similarity search, KNN search
Information Retrieval / Ranking	ranking systems, retrieval, NDCG, MRR, MAP, re-ranking, hybrid search, BM25	search ranking, recommendation system, matching system, relevance scoring, learning-to-rank
Strong Python	Python, production code quality, strong Python	Backend Python, Python engineering, pytest, type annotations
Evaluation Frameworks	NDCG, MRR, MAP, A/B testing, offline-to-online correlation, eval framework	model evaluation, ranking metrics, experiment design, online metrics

4.1.2 Preferred Skills (bonus points if present)

- LLM Fine-tuning: LoRA, QLoRA, PEFT, fine-tuning, instruction tuning, RLHF
- Learning to Rank: LambdaMART, XGBoost ranking, pointwise, pairwise, listwise rankers
- HR-Tech/Recruiting domain: talent platform, ATS, job matching, candidate scoring
- Distributed Systems: Kafka, Spark, Flink, large-scale inference, model serving
- Open-source contributions: GitHub contributions, ML projects, papers, talks

4.1.3 Disqualifying Patterns (hard filters)

- Entire career at ONLY: TCS, Infosys, Wipro, Accenture, Cognizant, Capgemini (no product company experience at all)
- Pure research roles: Academic lab, Research Scientist only, no production deployment
- Wrong domain only: Computer Vision, Speech, Robotics with zero NLP/IR/search experience
- Non-coding leadership only: VP Engineering, CTO with no hands-on coding in 18+ months
- AI experience < 12 months total: recent LangChain user without foundational ML background

4.1.4 Location Preferences

- PREFERRED: Pune, Noida (give max location score)
- GOOD: Hyderabad, Mumbai, Delhi NCR, Bangalore (give high location score)

- ACCEPTABLE: Other Tier-1 Indian cities, willing_to_relocate = true (medium score)
- NEUTRAL: Other India locations (moderate penalty)
- CASE-BY-CASE: Outside India, no visa sponsorship (apply penalty)

4.1.5 Experience Target

- Target: 5-9 years total (highest score in range 6-8 years)
- Still viable: 4-4.9 years if other signals are extremely strong
- Possible: 10-12 years if background shows progression (not stagnation)
- Risky: <4 years or >15 years (apply meaningful penalties)

4.2 Output Format of JD Parser

Store the parsed requirements as a Python dictionary with these keys:

```
requirements = {
    'hard_skills': [ {name, synonyms, weight} ],      # required skills with
    variants
    'preferred_skills': [ {name, synonyms, weight} ],
    'disqualifiers': { firm_list, research_only, cv_only, noncoding_lead,
recent_only },
    'location_scores': { 'Pune': 1.0, 'Noida': 1.0, 'Hyderabad': 0.85, ... },
    'experience': { 'ideal_min': 5, 'ideal_max': 9, 'peak_min': 6, 'peak_max':
8 },
    'jd_embedding': [...],      # pre-computed embedding of the full JD text
}
```

5. Data Loader & Preprocessor

5.1 Loading Strategy

The full candidates.jsonl is ~465 MB with exactly 100,000 lines. Loading all at once uses ~2-3 GB RAM. Use streaming to keep memory under control.

MEMORY STRATEGY: Stream candidates.jsonl line by line using ijson or simple json.loads per line. Process each candidate, compute score, store only (candidate_id, score, top_features) in memory. This reduces peak RAM from ~3GB to <500MB.

5.2 Data Normalisation Rules

Field	Issue to Handle	Normalisation Logic
skills[].name	Inconsistent casing, abbreviations	Lowercase and strip. Build alias map: 'bert' → 'transformers', 'gpt' → 'llm', 'faiss' → 'vector database', 'elasticsearch' → 'search'
location	Many formats: 'Bangalore', 'Bengaluru', 'Bengaluru, Karnataka'	Strip, lowercase, take first city part before comma. Map 'bengaluru' → 'bangalore'
career_history[].description	Free text, varies in length	Combine all descriptions into one string per candidate for embedding
career_history[].company	Case variations	Lowercase, strip. Build consulting firm list with case-insensitive matching
years_of_experience	Could be float	Round to 1 decimal for display; use raw float for calculations
redrob_signals.last_active_date	Date string	Compute days_since_active = (today - last_active_date).days at processing time
redrob_signals.offer_acceptance_rate	-1 if no history	Convert -1 to None; treat None as neutral (0.5) in scoring
redrob_signals.github_activity_score	-1 if no GitHub	Convert -1 to None; treat None as neutral in scoring

5.3 Candidate Text for Embedding

For the semantic matching component, you need to build one text string per candidate that represents their full professional identity. Build it as:

```
candidate_text = f"{headline}. {summary}. "  
for role in career_history:  
    candidate_text += f"{title} at {company}: {description}. "
```

```
candidate_text += f"Skills: {'', ' '.join(skill.name for skill in  
skills[:10])}."
```

Pre-compute embeddings for all 100K candidates OFFLINE using sentence-transformers. Store as numpy array. At ranking time, compute cosine similarity between JD embedding and candidate embedding.

6. Honeypot Detector — Identifying Impossible Profiles

CRITICAL WARNING: If >10% of your top 100 are honeypots, your submission is AUTOMATICALLY DISQUALIFIED regardless of score. This is a hard rule. Run this check before scoring.

6.1 Honeypot Detection Rules

Apply ALL these checks. If ANY flag is True, mark candidate as honeypot (relevance = 0):

Rule 1: Experience Timeline Impossibility

For each career_history entry, check:

- $(\text{end_date} - \text{start_date}) / 30$ differs from duration_months by more than 6 months → FLAG
- start_date before candidate's plausible birth year (years_of_experience implies graduation ~22 years old) → FLAG
- Career start date is more than $(\text{years_of_experience} + 4)$ years ago → FLAG
- Two career entries have overlapping date ranges (start/end dates collide) → FLAG

Rule 2: Skill Proficiency Impossibility

For each skill entry, check:

- proficiency == 'expert' AND duration_months < 6 AND endorsements == 0 AND skill not in skill_assessment_scores → FLAG
- proficiency == 'advanced' AND duration_months == 0 → FLAG
- A candidate claims 'expert' in 8+ different skills simultaneously → SUSPICIOUS (soft flag, reduce trust 30%)

Rule 3: Education Timeline Impossibility

For each education entry, check:

- end_year < start_year → FLAG
- A degree that normally takes 4 years shows end_year - start_year < 1 → FLAG
- Education overlaps with earliest career_history date in an impossible way → FLAG

Rule 4: Overall Data Consistency

- years_of_experience claims 8 years but sum of all career_history duration_months is less than 36 → FLAG
- profile.current_company matches career_history entry where is_current=true but start_date is more than 2× the company's likely age for unusual company names → SUSPICIOUS
- Profile completeness score > 95 but many fields are generic/templated → SUSPICIOUS (reduce trust, not full honeypot)

6.2 Honeypot Scoring

Do not use a binary flag only. Use a honey pot confidence score:

Honey pot confidence	Action
1.0 (hard impossibility)	Set final_score = 0. Never include in top 100.
0.7-0.9 (multiple suspicious signals)	Multiply final_score by 0.1. Very unlikely to rank in top 100.
0.4-0.7 (soft concerns)	Multiply final_score by 0.5. Monitor.
<0.4 (minor anomaly)	No penalty. Log for review.

7. Feature Engineering — All Features to Compute

For each candidate, compute these features. All are on a 0-1 scale before weighting.

7.1 Skill Match Score (0-1)

Step 1: Build the JD Core Skills List with Weights

Skill Category	Weight in Score	Examples
Embeddings / Dense Retrieval	0.20	sentence-transformers, OpenAI Ada, BGE, E5, SBERT, bi-encoder
Vector DB / ANN Search	0.20	FAISS, Pinecone, Qdrant, Weaviate, Milvus, Elasticsearch, OpenSearch
Retrieval & Ranking Systems	0.20	BM25, hybrid search, re-ranking, learning-to-rank, retrieval pipeline
LLMs & NLP	0.15	LLM fine-tuning, NLP, transformers, BERT, RAG, LoRA, QLoRA, PEFT
Evaluation & ML Engineering	0.15	NDCG, MRR, MAP, A/B testing, eval framework, model quality metrics
Python & Engineering	0.10	Python, production code, FastAPI, APIs, system design

Step 2: For Each Skill Category, Compute Category Score

```
def compute_category_score(candidate_skills, category_keywords):
    best_match = 0
    for skill in candidate_skills:
        if skill.name_normalised in category_keywords:
            trust = compute_skill_trust(skill) # see below
            best_match = max(best_match, trust)
    return best_match
```

Step 3: Compute Skill Trust (the key formula)

Trust prevents keyword stuffers. A claim without evidence = low trust.

```
proficiency_weights = {beginner: 0.25, intermediate: 0.55, advanced: 0.80,
                        expert: 1.0}

def compute_skill_trust(skill, assessment_scores):
    base = proficiency_weights[skill.proficiency]
    # Duration factor: scales from 0 (0 months) to 1.0 (24+ months)
    duration_factor = min(1.0, skill.duration_months / 24)
    # Endorsement bonus: adds up to 0.3 for well-endorsed skills
```

```

endorsement_bonus = min(0.3, skill.endorsements / 50 * 0.3)
# Assessment bonus: if platform tested this skill
assessment_bonus = 0
if skill.name in assessment_scores:
    score = assessment_scores[skill.name]
    assessment_bonus = (score / 100) * 0.2 # up to 0.2 bonus
trust = base * duration_factor + endorsement_bonus + assessment_bonus
return min(1.0, trust)

```

WHY THIS MATTERS: Marketing Manager claims 'expert Python' with 0 months duration, 0 endorsements, no assessment → $\text{trust} = 0.0 \times 0.0 = 0.0$. A real ML engineer with 'advanced Python', 36 months, 25 endorsements → $\text{trust} = 0.80 \times 1.0 + 0.15 = 0.95$.

Step 4: Final Skill Match Score

```

skill_match_score = sum(category_weight * category_score for each category)
# Then add preferred skills bonus (capped at +0.1)
preferred_bonus = min(0.1, count_of_preferred_skills_present * 0.025)
skill_score = min(1.0, skill_match_score + preferred_bonus)

```

Step 5: Text Semantic Bonus (catches plain-language fits)

```

# Pre-computed offline:
jd_embedding = embed(job_description_text)
candidate_embedding = embed(candidate_summary + all career descriptions)

semantic_score = cosine_similarity(jd_embedding, candidate_embedding) # 0 to 1
# Blend: 70% keyword-trust score + 30% semantic score
final_skill_score = 0.7 * skill_score + 0.3 * semantic_score

```

7.2 Career Quality Score (0-1)

Step 1: Product Company Detection

```

CONSULTING_FIRMS = ['tcs', 'tata consultancy', 'infosys', 'wipro',
                    'accenture',
                    'cognizant', 'capgemini', 'hcl', 'tech mahindra',
                    'mphasis']

def classify_company_type(company_name):
    if company_name.lower() in CONSULTING_FIRMS:
        return 'consulting'
    elif industry in ['IT Services', 'Outsourcing', 'BPO']:

```

```

        return 'services'
    else:
        return 'product'

all_types = [classify_company_type(role) for role in career_history]
if all(t == 'consulting' for t in all_types):
    company_type_score = 0.0 # Disqualifier
elif 'product' in all_types:
    product_months = sum(role.duration_months for role if type='product')
    company_type_score = min(1.0, product_months / 36) # 3+ years at product
    = max
else:
    company_type_score = 0.4 # Mixed: services but not pure consulting

```

Step 2: Production Deployment Signal (from descriptions)

Search career_history descriptions for production deployment evidence:

- Positive keywords: 'deployed', 'production', 'shipped', 'launched', 'served', 'at scale', 'real users', 'latency', 'throughput', 'SLA', 'monitoring', 'rollout'
- Score: 0.1 per keyword found (capped at 1.0 across all roles)
- Multiply by role_weight (current role = 2×, previous roles = 1×)

Step 3: Career Trajectory Analysis

Check for upward progression in titles (Engineer → Senior Engineer → Lead):

- Titles moving upward over time: +0.2 bonus
- Flat trajectory (same title across 10 years): no bonus
- Downward movement: mild concern (-0.1)
- Title seniority at current role: Junior→0.3, Mid→0.6, Senior→0.9, Lead/Principal→0.95

Step 4: Industry Relevance

- Tech / Software / AI / Data: 1.0
- FinTech / EdTech / HealthTech / E-commerce: 0.8 (product company, digital)
- Manufacturing / FMCG with IT role: 0.6
- Pure consulting / IT services: 0.4

Final Career Quality Score

```

career_quality_score = (
    0.35 * company_type_score +
    0.25 * production_signal_score +
    0.20 * seniority_score +
    0.20 * industry_relevance_score
)

```

7.3 Experience Score (0-1)

```
yoe = candidate.profile.years_of_experience

if 6 <= yoe <= 8:      experience_score = 1.0    # Perfect sweet spot
elif 5 <= yoe < 6:     experience_score = 0.90
elif 8 < yoe <= 9:     experience_score = 0.90
elif 4 <= yoe < 5:     experience_score = 0.75
elif 9 < yoe <= 12:    experience_score = 0.80
elif 3 <= yoe < 4:     experience_score = 0.55
elif 12 < yoe <= 15:   experience_score = 0.65
elif yoe > 15:         experience_score = 0.45
else:                 experience_score = 0.30    # <3 years
```

7.4 Location Score (0-1)

```
LOCATION_MAP = {
    'pune': 1.00, 'noida': 1.00,
    'delhi': 0.90, 'delhi ncr': 0.90, 'gurgaon': 0.90, 'gurugram': 0.90,
    'hyderabad': 0.90, 'mumbai': 0.85, 'bangalore': 0.85, 'bengaluru': 0.85,
    'chennai': 0.80, 'kolkata': 0.75, 'ahmedabad': 0.75,
    # other India: 0.60
    # outside India: 0.30 base
}

base_location_score = LOCATION_MAP.get(location_normalised, 0.60)
# If outside India but willing_to_relocate: no extra penalty (already priced in)
# If outside India and NOT willing_to_relocate: location_score = 0.15
if country != 'India' and not willing_to_relocate:
    location_score = 0.15
else:
    location_score = base_location_score
```

7.5 Education Score (0-1)

Education is a weaker signal for this role (JD is experience-focused). Weight accordingly.

```
tier_scores = {'tier_1': 1.0, 'tier_2': 0.80, 'tier_3': 0.65, 'tier_4': 0.50,
               'unknown': 0.55}

# Field of study bonus
```

```
relevant_fields = ['computer science', 'cs', 'data science', 'ai', 'machine  
learning',  
                  'statistics', 'mathematics', 'engineering']  
field_bonus = 0.1 if any field in relevant_fields else 0.0  
  
education_score = tier_scores[best_education_tier] + field_bonus
```

8. Behavioral Signal Scorer

8.1 The Behavioral Multiplier Approach

Behavioral signals do not replace the base score — they MULTIPLY it. Even a perfect skills match means nothing if the candidate is unreachable. The multiplier ranges from 0.1 (worst case: totally inactive) to 1.3 (best case: highly engaged, responsive, ready to move).

8.2 Availability Multiplier (most important)

```
def availability_multiplier(signals):
    multiplier = 1.0
    days_inactive = (today - signals.last_active_date).days

    # 1. Open to work flag
    if not signals.open_to_work_flag:
        multiplier *= 0.50

    # 2. Recency of activity
    if days_inactive > 180: multiplier *= 0.40
    elif days_inactive > 90: multiplier *= 0.65
    elif days_inactive > 30: multiplier *= 0.85
    else: multiplier *= 1.00

    # 3. Recruiter response rate
    rrr = signals.recruiter_response_rate
    if rrr < 0.05: multiplier *= 0.50
    elif rrr < 0.15: multiplier *= 0.75
    elif rrr < 0.30: multiplier *= 0.90
    elif rrr > 0.60: multiplier *= 1.10 # small bonus

    return min(1.30, max(0.10, multiplier))
```

8.3 Notice Period Modifier

```
def notice_period_modifier(notice_days):
    if notice_days <= 30: return 1.10 # JD wants sub-30
    elif notice_days <= 60: return 1.00
    elif notice_days <= 90: return 0.92
    elif notice_days <= 120: return 0.85
    else: return 0.78 # 120+ days is rough
```


8.4 Platform Quality Signals

```
def platform_quality_score(signals):
    score = 0.0

    # GitHub activity (important for AI/ML engineer)
    if signals.github_activity_score == -1:
        github_contrib = 0.0 # No GitHub = neutral
    else:
        github_contrib = signals.github_activity_score / 100 * 0.3

    # Interview completion (reliability signal)
    icr = signals.interview_completion_rate
    interview_score = icr * 0.2

    # Profile completeness
    completeness_score = max(0, (signals.profile_completeness_score - 50) /
50) * 0.1

    # Verification bonus
    verify_bonus = 0.05 if (signals.verified_email and
signals.verified_phone) else 0.0

    score = github_contrib + interview_score + completeness_score +
verify_bonus
    return min(0.5, score)
```

8.5 Final Behavioral Score

```
behavioral_score = (
    availability_multiplier *
    notice_period_modifier
) + platform_quality_score
behavioral_score = min(1.5, max(0.05, behavioral_score))
```

RULE: A candidate with perfect skills but `last_active_date = 7 months ago` and `recruiter_response_rate = 0.02` gets `availability_multiplier = 0.40 × 0.50 = 0.20`. Their final score is cut to 20% of their base score. They should NOT appear in top 20.

9. Final Scoring Engine — Composite Formula

9.1 Component Weights

Component	Weight	Rationale
Skills Match Score	35%	Core technical fit — most important dimension for this specific role
Career Quality Score	30%	Product company + production experience + seniority — JD explicitly cares about this
Experience Score	10%	Years of experience — important but not a hard filter
Behavioral Multiplier	Applied as multiplier after base	Availability — candidate may be perfect but unreachable
Location Score	10%	JD strongly prefers India / Pune / Noida
Education Score	5%	Mild signal — experience-first role
Honeypot Penalty	Hard zeroing	Any impossible profile = 0 score
Disqualifier Penalty	Strong reduction	Consulting-only, research-only, wrong domain = 0.1-0.4 multiplier

9.2 Composite Score Formula

```
base_score = 0.35×skill + 0.30×career + 0.10×experience + 0.10×location + 0.05×education
```

```
final_score = base_score × behavioral_multiplier × disqualifier_penalty
```

```
# Disqualifier penalties — these are multiplicative
if candidate_is_honeypot:           disqualifier_penalty = 0.0
elif all_consulting_no_product:     disqualifier_penalty = 0.1
elif all_pure_research:             disqualifier_penalty = 0.2
elif cv_speech_only_no_nlp:         disqualifier_penalty = 0.3
elif noncoding_leadership:          disqualifier_penalty = 0.5
elif recent_only_langchain:         disqualifier_penalty = 0.4
else:                               disqualifier_penalty = 1.0 # no penalty
```

9.3 Score Normalization

```
# After computing all 100K scores:
max_score = max(all_scores)
# Normalize so top candidate gets score = 1.0
```

```
# But ONLY if max_score > 0
normalised_scores = [s / max_score for s in all_scores]

# Sort descending, take top 100
top_100 = sorted(zip(candidate_ids, normalised_scores), key=lambda x: x[1],
reverse=True)[:100]

# Ensure scores are non-increasing (handle ties by candidate_id ascending)
# Equal scores: candidate_id ascending = correct tie-break per
submission_spec
```

10. Reasoning Generator — Per-Candidate 1-2 Sentence Reasoning

10.1 Rules for Good Reasoning

From submission_spec.docx, Stage 4 evaluators look for:

Requirement	Good Example	Bad Example
Reference actual profile data — do NOT hallucinate	'7.4 yrs at product companies; advanced FAISS + embedding drift experience'	'Strong candidate with excellent skills'
1-2 sentences only	'ML Engineer, 6.3yrs; built dense retrieval at Flipkart; Pune-based, 30-day notice.'	A paragraph of generic praise
Mention the key reason they ranked where they did	'Ranked #4 over stronger skills candidates due to 150-day notice period.'	Just listing features without saying WHY they rank here
Note significant concerns honestly	'Strong fit but last active 95 days ago and recruiter response rate 0.12 raises availability concern.'	Ignoring all red flags and being purely positive
Be specific, not templated	Each candidate gets unique text referencing their actual data	All candidates get 'X yrs; Y AI skills; response rate Z' — this is what gets penalized

10.2 Reasoning Logic per Rank Tier

Tier 1: Ranks 1-10 (must be very specific and accurate)

Template: [Top strength from career: specific company/project + skill]. [Location + availability confirmation if positive. If any concern: name it explicitly.]

```
Example: 'Senior ML Eng, 7.1 yrs at product cos (Meesho, Swiggy); built dense retrieval + FAISS at scale with embedding drift handling; Bangalore-based, 30-day notice, response rate 0.74.'
```

Tier 2: Ranks 11-50 (solid reasons, note any trade-offs)

Template: [Role + years + key matching skills from descriptions]. [One trade-off or note.]

```
Example: 'Applied ML Eng, 5.8 yrs; strong NLP + vector search background at Juspay; consulting stint early-career but last 3 yrs at product cos; hybrid preference, 45-day notice.'
```

Tier 3: Ranks 51-100 (acknowledge they are marginal, be honest)

Template: [What is strong]. [What is missing or concerning.]

```
Example: 'Data Scientist, 6.2 yrs; recommendation system experience but
limited retrieval/ranking
depth; adjacent fit included in top 100 as lower-confidence candidate.'
```

10.3 Reasoning Builder Logic

```
def build_reasoning(candidate, score, rank, features):
    parts = []

    # Sentence 1: Who they are and top strength
    title = candidate.profile.current_title
    yoe = candidate.profile.years_of_experience
    top_skills = get_top_matched_skills(features, n=3) # top 3 matching
skills
    production_co = get_best_product_company(career_history)
    parts.append(f'{title}, {yoe}yrs; {top_skills} at {production_co}.')

    # Sentence 2: Location + availability OR concern flag
    location = candidate.profile.location
    notice = candidate.signals.notice_period_days
    if features.behavioral_multiplier < 0.5:
        days_inactive = features.days_inactive
        parts.append(f'Concern: last active {days_inactive}d ago;
availability uncertain.')
    else:
        parts.append(f'{location}-based; {notice}-day notice; response rate
{rrr:.2f}.')

    return ' '.join(parts)
```

11. Submission Requirements — Exact Specs

11.1 CSV Output Format

Required columns in this exact order:

```
candidate_id,rank,score,reasoning
CAND_0042871,1,0.987,"Senior AI Eng, 7yrs; FAISS+embedding at scale; Pune-
based, 30-day notice."
CAND_0019884,2,0.973,"ML Eng 6yrs; built hybrid search at product co; strong
NLP background."
...
CAND_0007729,100,0.412,"Adjacent skills only; below cutoff but included as
final candidate."
```

Rule	Detail
Exactly 100 rows	Not 99, not 101. Exactly 100 data rows + 1 header row.
Ranks 1-100	Each rank appears exactly once. No rank = 0.
Scores non-increasing	score[rank=1] >= score[rank=2] >= ... >= score[rank=100]. Ties allowed.
Tie-breaking	If two candidates have same score: candidate_id ascending order.
No duplicate candidate_ids	Every CAND_XXXXXXX appears at most once.
candidate_ids must exist	Every id must be in candidates.jsonl. No invented ids.
UTF-8 encoding	Save CSV as UTF-8, not Latin-1.
Filename	Your registered participant ID + .csv (e.g. team_manish.csv)

11.2 Pre-submission Validation

Run this before every submission:

```
python validate_submission.py your_team_id.csv
```

Fix all errors it reports. Only submit if it prints 'Submission is valid.'

11.3 Submission Bundle — All 3 Parts Required

Part	What to Submit	Key Requirements
1. CSV File	The ranked 100 candidates	See 11.1 above
2. GitHub Repo	Full source code	README with single command: <code>python rank.py --candidates candidates.jsonl --out submission.csv</code>
3. Sandbox Demo	Working hosted environment	HuggingFace Spaces, Streamlit Cloud, Replit, Colab, or Docker — must accept <100 candidates and produce ranked CSV in <5 min on CPU

12. Pre-computation Strategy — Time Budget

12.1 Compute Constraint

HARD CONSTRAINT: The ranking step (candidates.jsonl → submission.csv) must complete in ≤5 minutes on CPU with 16GB RAM and NO network access. LLM API calls during ranking = DISQUALIFICATION.

12.2 What to Pre-Compute Offline

Pre-computation Task	When to Run	Output File	Approximate Size
Embed all 100K candidate texts using sentence-transformers (e.g. all-MiniLM-L6-v2)	Once, offline. Takes 20-40 min on CPU.	candidate_embeddings.npy	~400 MB (100K × 384 dims × 4 bytes)
Compute JD embedding	Once, offline. Takes 1 second.	jd_embedding.npy	<1 KB
Pre-compute all feature vectors for 100K candidates (all non-embedding features)	Once, offline. Takes 5-10 min.	candidate_features.pkl	~50-100 MB
Pre-compute honeypot flags	Once, offline. Takes 2-3 min.	honeypot_flags.json	<5 MB

12.3 Online (Ranking) Step — Within 5-Minute Budget

With pre-computation done, the online step only needs to:

1. Load candidate_embeddings.npy and jd_embedding.npy (seconds)
2. Load candidate_features.pkl (seconds)
3. Compute cosine similarities in numpy (vectorized, < 10 seconds for 100K)
4. Apply scoring formula to pre-computed features (< 30 seconds)
5. Sort, select top 100, generate reasoning, write CSV (< 30 seconds)

Total online time estimate: under 2 minutes. Well within the 5-minute budget.

12.4 Model Choice for Embeddings

Model	Pros	Cons	Recommendation
all-MiniLM-L6-v2	384 dims, very fast, good quality, 80MB	Not the strongest semantic quality	RECOMMENDED for this task — fast enough to fit budget
all-mpnet-base-v2	768 dims, better quality, 420MB	2× slower and larger	Good if you have time to test
BGE-small-en	Small, fast, trained on	Less general	Good alternative

Model	Pros	Cons	Recommendation
	retrieval tasks		
OpenAI text-embedding-ada-002	Best quality	Requires API call — BANNED during ranking step	Use only for offline pre-computation if you go this route

RECOMMENDATION: Use sentence-transformers all-MiniLM-L6-v2 for offline embedding of all 100K candidates. It is fast, free, CPU-compatible, and produces high-quality semantic matches for this type of professional text.

13. Full Implementation Checklist

Step-by-Step Build Order

Follow this order — each step builds on the previous:

Step	Task	File(s)	Test Condition
1	Set up project structure and requirements.txt (sentence-transformers, numpy, python-docx, pyyaml)	requirements.txt	pip install -r requirements.txt runs without errors
2	Build config.py with all skill lists, weights, firm lists, location maps, thresholds	config.py	Can import and read all values correctly
3	Build jd_parser.py to extract requirements from job_description.docx	jd_parser.py	requirements dict has all hard_skills, preferred_skills, disqualifiers populated
4	Build data_loader.py to stream candidates.jsonl	data_loader.py	Loads all 100K records, parses without error, yields correct dicts
5	Build honeypot_detector.py with all 4 check categories	honeypot_detector.py	Identifies at least some flagged candidates in sample_candidates.json
6	Run precompute_embeddings.py on all 100K candidates	precompute_embeddings.py	candidate_embeddings.npy is 100K rows, no NaN values
7	Build skill_scorer.py with trust formula and semantic blend	skill_scorer.py	ML Engineers score higher than Marketing Managers on same skills
8	Build career_analyzer.py with consulting detection and production signals	career_analyzer.py	TCS-only candidates score near 0 on career quality
9	Build behavioral_scorer.py with availability multiplier	behavioral_scorer.py	Inactive candidates (180d ago) get multiplier < 0.4
10	Build scoring_engine.py combining all features	scoring_engine.py	Output is float 0-1, non-increasing when sorted
11	Build reasoning_generator.py	reasoning_generator.py	Each candidate gets unique 1-2 sentence reasoning referencing actual profile data
12	Build rank.py as main entrypoint	rank.py	python rank.py --candidates sample_candidates.json --out test.csv runs in <1 min, produces valid CSV
13	Run validate_submission.py on output	—	'Submission is valid.' printed
14	Run on FULL candidates.jsonl,	—	Completes in <5 minutes

Step	Task	File(s)	Test Condition
	time it		
15	Manual review: check top 10 candidates make sense	—	Top 10 are ML/AI engineers at product companies with strong retrieval experience
16	Check honeypot rate in top 100	—	< 8 honeypots in top 100 (target: 0-5)
17	Set up Sandbox (HuggingFace Spaces or Streamlit)	sandbox/	Organizers can run it on sample_candidates.json and get ranked CSV
18	Fill submission_metadata.yaml	submission_metadata.yaml	All fields populated, declarations = true
19	Final validate and submit	—	3 submissions max — use them wisely

14. Traps & Edge Cases Quick Reference

Trap / Edge Case	Detection	Handling
Marketing Manager with full AI skills list	current_title in non-tech titles + skills list has LLM/embedding/Python	skill_trust $\approx$ 0 because career_history descriptions have no AI deployment evidence. Career quality = 0. Final score very low.
Honeypot: 8 yrs at 3-yr-old company	career_history duration_months >> plausible company age	Set final_score = 0. Never include in top 100.
Honeypot: expert skill, 0 months duration	proficiency='expert' AND duration_months < 6 AND endorsements=0	Set honeypot_confidence = 1.0. Final score = 0.
Inactive perfect candidate (180d+ no login)	days_since_active > 180	Availability multiplier $\times$ 0.40. Score cut by 60%.
Consulting-only career (Infosys 15 yrs)	All career_history companies in CONSULTING_FIRMS	Disqualifier penalty $\times$ 0.1. Nearly impossible to rank in top 100.
Plain-language fit (no keywords but real experience)	Semantic similarity to JD is high even though keywords are absent	Semantic blend (30% of skill score) catches these. Career quality is also high.
TCS candidate with prior product company experience	career_history has some consulting AND some product	No consulting penalty. Score on merit of product company experience.
AI Engineer title but only LangChain tutorials in descriptions	Title = AI Engineer but descriptions mention only 'used LangChain to call GPT'	semantic_score will be moderate but career quality penalty for recency and no pre-LLM experience applies.
Candidate outside India, willing_to_relocate = true	country != India, willing_to_relocate = true	Location score = 0.30 (penalty for outside India but no extra penalty for willing to relocate).
Very high salary expectation (100+ LPA)	expected_salary_range_inr_lpa.max > 80 for a Series A startup	Flag in reasoning only. Do not penalize — their skills might still justify ranking. Note: 'Salary expectation high for Series A.'

END OF IMPLEMENTATION BLUEPRINT

Dataset: 100,000 candidates | Target: Top 100 ranked with scores and reasoning

Scoring: 50% NDCG@10 + 30% NDCG@50 + 15% MAP + 5% P@10